

Talas App — API/Endpoint Documentation

Base URL: `/api/trpc` (tRPC) or `/api/*` (REST) **Auth:** Clerk (Bearer Token via session). All tRPC procedures use `protectedProcedure` unless noted.

Table of Contents

- 1. [User Router](#)
- 2. [Project Router](#)
- 3. [Comment Router](#)
- 4. [Follow Router](#)
- 5. [Bookmark Router](#)
- 6. [Like Project Router](#)
- 7. [Like Comment Router](#)
- 8. [Notification Router](#)
- 9. [Collaboration Router](#)
- 10. [Search Router](#)
- 11. [Category Router](#)
- 12. [Pin Router](#)
- 13. [REST API Endpoints](#)

1. User Router

All endpoints prefixed with `user.*` via tRPC.

1.1 Sync User with Database

Field	Value
Endpoint	<code>/api/trpc/user.syncWithSupabase</code>
Method	<code>mutation</code> (POST)
Auth	Bearer Token (Clerk Session)
Permission	Authenticated User
Description	Creates or verifies a user record in the database. If the user doesn't exist, creates a new user and associated <code>count_summary</code> . Generates a unique username from email.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id</code>	String	✓	Clerk user ID
<code>name</code>	String	✓	Display name
<code>email</code>	String (email)	✓	Valid email address
<code>auth_type</code>	String	✓	Auth provider (e.g. <code>"google"</code> , <code>"github"</code>)
<code>photo_profile</code>	String	✓	URL to profile photo

Success Response — `200 OK`

```
{
  "success": true,
  "message": "Successfully synced user with Supabase",
  "data": {
    "id": "user_2abc123",
    "username": "dev",
    "name": "Dev User",
    "email": "dev@example.com",
    "auth_type": "google",
    "photo_profile": "https://...",
    "created_at": "2026-05-04T10:00:00.000Z"
  }
}
```

Errors

Code	Message
<code>401 UNAUTHORIZED</code>	Authentication token missing or invalid
<code>500 INTERNAL_SERVER_ERROR</code>	Database sync failure

1.2 Get All Users

Field	Value
Endpoint	<code>/api/trpc/user.getAll</code>
Method	<code>query</code> (GET)
Auth	Bearer Token (Clerk Session)
Description	Returns a paginated list of all users with profile info.

Request Body (JSON) — Optional

Parameter	Type	Required	Default	Description
<code>limit</code>	Integer	✗	—	Max records to return
<code>offset</code>	Integer	✗	—	Records to skip

Success Response — 200 OK

```
{
  "success": true,
  "message": "Successfully get all users",
  "data": [
    {
      "username": "dev",
      "name": "Dev User",
      "bio": "Hello world",
      "photo_profile": "https://...",
      "instagram": null,
      "linkedin": null,
      "github": "https://github.com/dev",
      "gender": "MALE",
      "email_contact": "dev@example.com"
    }
  ]
}
```

1.3 Get User by ID

Field	Value
Endpoint	<code>/api/trpc/user.getById</code>
Method	<code>query</code> (GET)
Auth	Bearer Token (Clerk Session)
Description	Retrieves a single user by their ID, including count summary.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id</code>	String	✓	User ID

Success Response — 200 OK

```
{
  "success": true,
  "message": "Successfully get user",
  "data": {
    "id": "user_2abc123",
    "username": "dev",
    "name": "Dev User",
    "bio": "Hello",
    "photo_profile": "https://...",
    "instagram": null,
    "linkedin": null,
    "github": null,
    "gender": "MALE",
    "email_contact": "dev@example.com",
    "count_summary": {
      "count_project": 5,
      "count_follower": 12,
      "count_following": 8,
      "all_notif_read": true
    }
  }
}
```

1.4 Get User by Username

Field	Value
Endpoint	<code>/api/trpc/user.getByUsername</code>
Method	<code>query</code> (GET)
Auth	Bearer Token (Clerk Session)
Description	Retrieves a single user by username, including count summary.

Request Body (JSON)

Parameter	Type	Required	Description
<code>username</code>	String		Username

Success Response — Same shape as `getById`.

1.5 Update User Profile

Field	Value
Endpoint	<code>/api/trpc/user.update</code>
Method	<code>mutation</code> (POST)
Auth	Bearer Token (Clerk Session)
Description	Updates user profile fields. Validates username uniqueness if changed.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id</code>	String	✓	User ID
<code>data.username</code>	String	✗	New username (must be unique)
<code>data.name</code>	String	✗	Display name
<code>data.bio</code>	String	✗	User bio
<code>data.photo_profile</code>	String	✗	Profile photo URL
<code>data.instagram</code>	String	✗	Instagram handle
<code>data.linkedin</code>	String	✗	LinkedIn URL
<code>data.github</code>	String	✗	GitHub URL
<code>data.gender</code>	Enum: <code>MALE</code> , <code>FEMALE</code>	✗	Gender
<code>data.email_contact</code>	String	✗	Contact email

Errors

Code	Message
<code>500</code>	"Username sudah digunakan oleh pengguna lain." — Username taken

1.6 Get Photo Profile

Field	Value
Endpoint	<code>/api/trpc/user.getPhotoProfile</code>
Method	<code>query</code> (GET)
Auth	Bearer Token (Clerk Session)
Description	Returns only the profile photo URL for a user by ID or username.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id</code>	String	✗	User ID
<code>username</code>	String	✗	Username

1.7 Get All Followers

Field	Value
Endpoint	<code>/api/trpc/user.getAllFollower</code>
Method	<code>query</code> (GET)
Auth	Bearer Token (Clerk Session)
Description	Returns paginated list of followers for a user.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id_following</code>	String	✓	The user whose followers to list
<code>limit</code>	Integer	✗	Max results
<code>offset</code>	Integer	✗	Skip count

1.8 Get All Following

Field	Value
Endpoint	<code>/api/trpc/user.getAllFollowing</code>
Method	<code>query</code> (GET)
Auth	Bearer Token (Clerk Session)
Description	Returns paginated list of users that a user follows.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id_follower</code>	String	✓	The user whose following list to retrieve
<code>limit</code>	Integer	✗	Max results
<code>offset</code>	Integer	✗	Skip count

1.9 Get Bookmarked Projects

Field	Value
Endpoint	<code>/api/trpc/user.getBookmarked</code>
Method	<code>query</code> (GET) — Infinite Query
Auth	Bearer Token (Clerk Session)
Description	Returns cursor-paginated bookmarked projects for a user with interaction status.

Request Body (JSON)

Parameter	Type	Required	Default	Description
<code>id</code>	String	✓	—	User ID (bookmark owner)
<code>id_user</code>	String	✗	—	Viewer user ID (for interaction flags)
<code>limit</code>	Integer (1–100)	✗	12	Page size
<code>cursor</code>	String	✗	—	Pagination cursor

1.10 Get Select Collab Options

Field	Value
Endpoint	<code>/api/trpc/user.selectCollab</code>
Method	<code>query</code> (GET)
Auth	Bearer Token (Clerk Session)
Description	Searches users by name/username for collaborator selection, excluding self.

Request Body (JSON)

Parameter	Type	Required	Description
<code>query</code>	String	✓	Search term
<code>id_user</code>	String	✓	Current user ID (excluded)

1.11 Get Notifications (User)

Field	Value
Endpoint	<code>/api/trpc/user.getNotification</code>
Method	<code>query</code> (GET) — Infinite Query
Auth	Bearer Token (Clerk Session)
Description	Returns cursor-paginated notifications from the last 30 days.

Request Body (JSON)

Parameter	Type	Required	Default	Description
<code>id_user</code>	String	✓	—	User ID
<code>limit</code>	Integer (1–100)	✗	12	Page size
<code>cursor</code>	String	✗	—	Pagination cursor

1.12 Get Collaboration Requests

Field	Value
Endpoint	<code>/api/trpc/user.getRequestCollab</code>
Method	<code>query</code> (GET)
Auth	Bearer Token (Clerk Session)
Description	Returns pending collaboration requests for a user.

Request Body — `String` (user ID directly)

1.13 Get All User Projects

Field	Value
Endpoint	<code>/api/trpc/user.getAllProjects</code>
Method	<code>query</code> (GET) — Infinite Query
Auth	Bearer Token (Clerk Session)
Description	Returns cursor-paginated projects owned or collaborated on by a user. Can exclude pinned projects.

Request Body (JSON)

Parameter	Type	Required	Default	Description
<code>id_user</code>	String	✗	—	User ID
<code>limit</code>	Integer (1–100)	✗	50	Page size
<code>cursor</code>	String	✗	—	Cursor
<code>excludePinned</code>	Boolean	✗	—	Exclude pinned projects

1.14 Get Pinned Projects

Field	Value
Endpoint	<code>/api/trpc/user.getPinnedProjects</code>
Method	<code>query</code> (GET)
Auth	Bearer Token (Clerk Session)
Description	Returns all pinned projects for a user with interaction flags.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id_user</code>	String	☒	User ID

2. Project Router

All endpoints prefixed with `project.*` via tRPC.

2.1 Get One Project

Field	Value
Endpoint	<code>/api/trpc/project.getOne</code>
Method	<code>query</code> (GET)
Auth	Bearer Token (Clerk Session)
Description	Retrieves a single project by ID or slug. Archived projects visible only to owners. Includes bookmark/like status.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id</code>	String	☒	Project ID or slug
<code>id_user</code>	String	☒	Viewer user ID (for interaction flags)

Success Response — `200 OK`

```
{
  "success": true,
  "message": "Successfully get project",
  "data": {
    "id": "01HX...",
    "title": "My Project",
    "slug": "my-project",
    "content": "<p>Project description...</p>",
    "image1": "path/to/image.webp",
    "image2": null,
    "image3": null,
    "image4": null,
    "image5": null,
    "video": null,
    "count_likes": 5,
    "count_comments": 3,
    "is_archived": false,
    "link_figma": null,
    "link_github": "https://github.com/...",
    "created_at": "2026-05-04T10:00:00.000Z",
    "updated_at": "2026-05-04T10:00:00.000Z",
    "category": { "id": "...", "title": "Web", "slug": "web" },
    "project_user": [
      {
        "user": { "id": "...", "name": "Dev", "username": "dev", "photo_profile": "..." },
        "ownership": "OWNER"
      }
    ],
    "is_bookmarked": false,
    "is_liked": true
  }
}
```

2.2 Get All Projects

Field	Value
Endpoint	<code>/api/trpc/project.getAll</code>
Method	<code>query</code> (GET) — Infinite Query
Auth	Bearer Token (Clerk Session)
Description	Returns cursor-paginated non-archived projects with interaction flags.

Request Body (JSON)

Parameter	Type	Required	Default	Description
<code>id_user</code>	String	✗	—	Viewer user ID
<code>limit</code>	Integer (1–100)	✗	50	Page size
<code>cursor</code>	String	✗	—	Cursor

2.3 Get Archived Projects

Field	Value
Endpoint	<code>/api/trpc/project.getArchived</code>
Method	<code>query</code> (GET) — Infinite Query
Auth	Bearer Token (Clerk Session)
Description	Returns cursor-paginated archived projects owned by the user.

Request Body (JSON)

Parameter	Type	Required	Default	Description
<code>id_user</code>	String	✗	—	Owner user ID
<code>limit</code>	Integer (1–100)	✗	50	Page size
<code>cursor</code>	String	✗	—	Cursor

2.4 Check Slug Availability

Field	Value
Endpoint	<code>/api/trpc/project.checkSlug</code>
Method	<code>query</code> (GET)
Auth	Bearer Token (Clerk Session)
Description	Returns <code>true</code> if the slug is already taken, <code>false</code> if available.

Request Body — `String` (slug to check)

2.5 Create Project

Field	Value
Endpoint	<code>/api/trpc/project.create</code>
Method	<code>mutation</code> (POST)
Auth	Bearer Token (Clerk Session)
Description	Creates a new project. Auto-generates slug from title. Increments category and user project counts. Sends collaboration requests to collaborators.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id_user</code>	String	✓	Owner user ID
<code>id_category</code>	String	✓	Category ID
<code>title</code>	String	✓	Project title
<code>content</code>	String (min 1 char)	✓	HTML content
<code>is_archived</code>	Boolean	✗	Default: <code>false</code>
<code>image1</code> – <code>image5</code>	Any	✗	Image paths
<code>video</code>	Any	✗	Video path
<code>collaborators</code>	Array of <code>{id, name, username, photo_profile?}</code>	✗	Collaborator list
<code>link_figma</code>	String (URL)	✗	Figma link
<code>link_github</code>	String (URL)	✗	GitHub link

2.6 Edit Project

Field	Value
Endpoint	<code>/api/trpc/project.edit</code>
Method	<code>mutation</code> (POST)
Auth	Bearer Token (Clerk Session)
Permission	Project Owner only
Description	Updates project fields. Re-generates slug if title changes. Updates category counts if category changes.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id</code>	String	✓	Project ID
<code>id_user</code>	String	✓	Owner user ID (for verification)
<code>id_category</code>	String	✗	New category
<code>title</code>	String	✗	New title
<code>content</code>	String	✗	New content
<code>link_figma</code>	String (URL)	✗	Figma link
<code>link_github</code>	String (URL)	✗	GitHub link

Errors

Code	Message
<code>500</code>	"Project not found or access denied."

2.7 Delete Project

Field	Value
Endpoint	<code>/api/trpc/project.delete</code>
Method	<code>mutation</code> (POST)
Auth	Bearer Token (Clerk Session)
Permission	Project Owner only
Description	Permanently deletes a project, its comments, and associated images from storage. Decrements category and user counts.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id</code>	String	✓	Project ID
<code>id_user</code>	String	✓	Owner user ID

2.8 Get Project Comments

Field	Value
Endpoint	<code>/api/trpc/project.getComments</code>
Method	<code>query</code> (GET)
Auth	Bearer Token (Clerk Session)
Description	Returns threaded comments for a project (max 1 level of nesting). Includes like status.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id</code>	String	✓	Project ID
<code>id_user</code>	String	✗	Viewer user ID (for <code>is_liked</code> flag)

2.9 Archive Project

Field	Value
Endpoint	<code>/api/trpc/project.archive</code>
Method	<code>mutation</code> (POST)
Auth	Bearer Token (Clerk Session)
Permission	Project Owner only
Description	Sets a project as archived. Decrements category and user project counts.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id</code>	String	✓	Project ID
<code>id_user</code>	String	✓	Owner user ID

2.10 Unarchive Project

Field	Value
Endpoint	<code>/api/trpc/project.unarchive</code>
Method	<code>mutation</code> (POST)
Auth	Bearer Token (Clerk Session)
Permission	Project Owner only
Description	Publishes an archived project. Increments category and user project counts.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id</code>	String	✓	Project ID
<code>id_user</code>	String	✓	Owner user ID

3. Comment Router

All endpoints prefixed with `comment.*` via tRPC.

3.1 Create Comment

Field	Value
Endpoint	<code>/api/trpc/comment.create</code>
Method	<code>mutation</code> (POST)
Auth	Bearer Token (Clerk Session)
Description	Creates a comment or reply on a project. Replies are limited to one level deep. Sends notifications to project owners, collaborators, and parent comment authors. Increments project comment count.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id_project</code>	String	✓	Project ID to comment on
<code>content</code>	String (1–1000 chars)	✓	Comment text
<code>parent_id</code>	String	null	✓

Success Response — `200 OK`

```
{
  "success": true,
  "message": "Successfully comment",
  "data": {
    "id": "01HX...",
    "id_project": "...",
    "id_user": "...",
    "content": "Great project!",
    "parent_id": null,
    "created_at": "2026-05-04T10:00:00.000Z",
    "updated_at": "2026-05-04T10:00:00.000Z"
  },
  "count_likes": 4
}
```

Errors

Code	Error	Description
400 BAD_REQUEST	—	Project ID and content are required, or reply depth > 1 level
401 UNAUTHORIZED	—	User not authenticated
404 NOT_FOUND	—	Parent comment not found / Project has no owner
500 INTERNAL_SERVER_ERROR	—	Database error

3.2 Edit Comment

Field	Value
Endpoint	/api/trpc/comment.edit
Method	mutation (POST)
Auth	Bearer Token (Clerk Session)
Permission	Comment author only
Description	Edits an existing comment's content. Only the comment author can edit.

Request Body (JSON)

Parameter	Type	Required	Description
id	String	✓	Comment ID
content	String (1–1000 chars)	✓	Updated text
id_user	String	✓	Author user ID (verified against session)

Errors

Code	Error	Description
401 UNAUTHORIZED	—	Not authenticated
403 FORBIDDEN	—	Not the comment author
404 NOT_FOUND	—	Comment not found

3.3 Delete Comment

Field	Value
Endpoint	/api/trpc/comment.deleteById
Method	mutation (POST)
Auth	Bearer Token (Clerk Session)
Permission	Comment author only
Description	Deletes a comment and all its descendant replies. Decrements project comment count by total deleted.

Request Body (JSON)

Parameter	Type	Required	Description
id	String	✓	Comment ID
id_user	String	✓	Author user ID
id_project	String	✓	Project ID (to update count)

4. Follow Router

All endpoints prefixed with `follow.*` via tRPC.

4.1 Follow User

Field	Value
Endpoint	/api/trpc/follow.following
Method	mutation (POST)
Auth	Bearer Token (Clerk Session)
Description	Creates a follow relationship. Increments follower/following counts. Sends a <code>FOLLOW</code> notification to the followed user.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id_follower</code>	String	✓	Follower user ID
<code>id_following</code>	String	✓	User to follow

Errors

Code	Error	Description
<code>404 NOT_FOUND</code>	—	One or both users not found
<code>409 CONFLICT</code>	—	Already following this user

4.2 Unfollow User

Field	Value
Endpoint	<code>/api/trpc/follow.unfollowing</code>
Method	<code>mutation</code> (POST)
Auth	Bearer Token (Clerk Session)
Description	Removes a follow relationship. Decrements follower/following counts.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id_follower</code>	String	✓	Follower user ID
<code>id_following</code>	String	✓	User to unfollow

Errors

Code	Error	Description
<code>404 NOT_FOUND</code>	—	Follow relationship not found

4.3 Check Is Following

Field	Value
Endpoint	<code>/api/trpc/follow.checkIsFollowing</code>
Method	<code>query</code> (GET)
Auth	Bearer Token (Clerk Session)
Description	Checks if user A is following user B.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id_follower</code>	String	✓	Follower user ID
<code>id_following</code>	String	✓	Target user ID

Success Response — **200 OK**

```
{ "isFollowing": true }
```

5. Bookmark Router

All endpoints prefixed with `bookmark.*` via tRPC.

5.1 Create Bookmark

Field	Value
Endpoint	<code>/api/trpc/bookmark.create</code>
Method	<code>mutation</code> (POST)
Auth	Bearer Token (Clerk Session)
Description	Bookmarks a project for a user.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id_user</code>	String	✓	User ID
<code>id_project</code>	String	✓	Project ID

Errors

Code	Error	Description
409 CONFLICT	—	Bookmark already exists

5.2 Delete Bookmark

Field	Value
Endpoint	<code>/api/trpc/bookmark.delete</code>
Method	<code>mutation</code> (POST)
Auth	Bearer Token (Clerk Session)
Description	Removes a user's bookmark on a project.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id_user</code>	String	✓	User ID
<code>id_project</code>	String	✓	Project ID

6. Like Project Router

All endpoints prefixed with `likeProject.*` via tRPC.

6.1 Like Project

Field	Value
Endpoint	<code>/api/trpc/likeProject.like</code>
Method	<code>mutation</code> (POST)
Auth	Bearer Token (Clerk Session)
Description	Likes a project. Increments project like count. Sends <code>LIKE_PROJECT</code> notifications to project owner and collaborators.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id_user</code>	String	✓	Liker user ID
<code>id_project</code>	String	✓	Project ID

Success Response — `200 OK`

```
{
  "success": true,
  "message": "Project liked successfully",
  "count_likes": 6
}
```

Errors

Code	Error	Description
404 NOT_FOUND	—	Project or user not found
409 CONFLICT	—	Already liked this project

6.2 Unlike Project

Field	Value
Endpoint	/api/trpc/likeProject.unlike
Method	mutation (POST)
Auth	Bearer Token (Clerk Session)
Description	Removes a like from a project. Decrements project like count.

Request Body (JSON)

Parameter	Type	Required	Description
id_user	String	✓	User ID
id_project	String	✓	Project ID

Errors

Code	Error	Description
404 NOT_FOUND	—	Like not found

7. Like Comment Router

All endpoints prefixed with `likeComment.*` via tRPC.

7.1 Like Comment

Field	Value
Endpoint	<code>/api/trpc/likeComment.like</code>
Method	<code>mutation</code> (POST)
Auth	Bearer Token (Clerk Session)
Description	Likes a comment. Increments comment like count. Sends <code>LIKE_COMMENT</code> notification to comment author if not self.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id_user</code>	String	✓	Liker user ID
<code>id_comment</code>	String	✓	Comment ID

Success Response — `200 OK`

```
{
  "success": true,
  "message": "Comment liked successfully",
  "data": { "id": "...", "id_user": "...", "id_comment": "..."},
  "count_like": 3
}
```

Errors

Code	Error	Description
<code>404 NOT_FOUND</code>	—	Comment not found
<code>409 CONFLICT</code>	—	Already liked

7.2 Unlike Comment

Field	Value
Endpoint	<code>/api/trpc/likeComment.unlike</code>
Method	<code>mutation</code> (POST)
Auth	Bearer Token (Clerk Session)
Description	Removes a like from a comment. Decrements comment like count.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id_user</code>	String	✓	User ID
<code>id_comment</code>	String	✓	Comment ID

Errors

Code	Error	Description
<code>404 NOT_FOUND</code>	—	Like not found

8. Notification Router

All endpoints prefixed with `notification.*` via tRPC.

8.1 Create Notification

Field	Value
Endpoint	<code>/api/trpc/notification.create</code>
Method	<code>mutation</code> (POST)
Auth	Bearer Token (Clerk Session)
Description	Creates a notification for a user.

Request Body (JSON)

Parameter	Type	Required	Default	Description
<code>id_user</code>	String	✓	—	Target user ID
<code>title</code>	String	✓	—	Notification message
<code>is_read</code>	Boolean	✗	<code>false</code>	Read status
<code>type</code>	Enum	✓	—	One of: <code>FOLLOW</code> , <code>LIKE_PROJECT</code> , <code>LIKE_COMMENT</code> , <code>COMMENT_PROJECT</code> , <code>COMMENT</code> , <code>REPLY_COMMENT</code> , <code>COLLABORATION</code>

Success Response — `200 OK`

```
{
  "success": true,
  "message": "Successfully created notification",
  "data": {
    "id": "01HX...",
    "title": "dev liked your project",
    "created_at": "2026-05-04T10:00:00.000Z",
    "is_read": false,
    "type": "LIKE_PROJECT"
  }
}
```

Errors

Code	Error	Description
404 NOT_FOUND	—	User not found

8.2 Mark All Notifications as Read

Field	Value
Endpoint	/api/trpc/notification.makeReaded
Method	mutation (POST)
Auth	Bearer Token (Clerk Session)
Description	Marks all unread notifications as read. Updates all_notif_read flag in count_summary.

Request Body (JSON)

Parameter	Type	Required	Description
id_user	String	✓	User ID

Success Response — 200 OK

```
{
  "success": true,
  "message": "All notifications marked as read",
  "count": 5
}
```


8.3 Get Unread Notification Status

Field	Value
Endpoint	<code>/api/trpc/notification.getIsUnread</code>
Method	<code>query</code> (GET)
Auth	Bearer Token (Clerk Session)
Description	Returns whether the user has any unread notifications or pending collaboration requests.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id_user</code>	String		User ID

Success Response — `200 OK`

```
{
  "success": true,
  "message": "Unread notifications count retrieved successfully",
  "data": true
}
```

9. Collaboration Router

All endpoints prefixed with `collaboration.*` via tRPC.

9.1 Accept Collaboration

Field	Value
Endpoint	<code>/api/trpc/collaboration.accept</code>
Method	<code>mutation</code> (POST)
Auth	Bearer Token (Clerk Session)
Description	Accepts a pending collaboration request. Updates status to <code>ACCEPTED</code> . Sends a <code>COLLABORATION</code> notification to the project owner.

Request Body — `String` (ProjectUser ID)

Success Response — `200 OK`

```
{
  "success": true,
  "message": "You have accepted the collaboration",
  "data": {
    "id": "01HX...",
    "id_project": "...",
    "ownership": "COLLABORATOR",
    "collabStatus": "ACCEPTED"
  }
}
```

Errors

Code	Error	Description
404 NOT_FOUND	—	Collaboration not found

9.2 Reject Collaboration

Field	Value
Endpoint	/api/trpc/collaboration.reject
Method	mutation (POST)
Auth	Bearer Token (Clerk Session)
Description	Rejects and deletes a pending collaboration request. Sends a COLLABORATION notification to the project owner.

Request Body — **String** (ProjectUser ID)

Errors

Code	Error	Description
404 NOT_FOUND	—	Collaboration not found

10. Search Router

All endpoints prefixed with **search.*** via tRPC.

10.1 Search (Projects, Users, Categories)

Field	Value
Endpoint	<code>/api/trpc/search.search</code>
Method	<code>query</code> (GET) — Infinite Query
Auth	Bearer Token (Clerk Session)
Description	Unified search endpoint. Searches projects by title, users by name/username, or categories by title. Supports cursor pagination and category filtering for projects.

Request Body (JSON)

Parameter	Type	Required	Default	Description
<code>search</code>	String (max 100)	✗	—	Search query
<code>type</code>	Enum: <code>PROJECT</code> , <code>USER</code> , <code>CATEGORY</code>	✓	—	Search type
<code>category</code>	String	✗	—	Category slug filter (projects only). Use <code>"__all__"</code> for all.
<code>id_user</code>	String	✓	—	Viewer user ID
<code>limit</code>	Integer (1–100)	✗	50	Page size
<code>cursor</code>	String	✗	—	Cursor

Success Response (PROJECT) — `200 OK`

```
{
  "success": true,
  "message": "Successfully get projects",
  "data": [
    {
      "id": "...",
      "title": "...",
      "slug": "...",
      "is_bookmarked": false,
      "is_liked": true,
      "category": { "id": "...", "title": "...", "slug": "..." },
      "project_user": [{ "user": { "...", "ownership": "OWNER" } }]
    }
  ],
  "nextCursor": "01HX..."
}
```

Success Response (USER)

```
{
  "success": true,
  "message": "Successfully get users",
  "data": [
    {
      "id": "...",
      "name": "Dev",
      "username": "dev",
      "photo_profile": "...",
      "github": "...",
      "count_summary": { "count_project": 5, "count_follower": 12, "count_following": 8 }
    }
  ],
  "nextCursor": "dev-123"
}
```

Errors

Code	Error	Description
400 BAD_REQUEST	—	Unsupported search type

10.2 Get Popular Posts

Field	Value
Endpoint	/api/trpc/search.getPopularPost
Method	query (GET)
Auth	Bearer Token (Clerk Session)
Description	Returns the most popular (highest popularity_score) non-archived project per category using a raw SQL window function.

Request Body (JSON)

Parameter	Type	Required	Description
id_user	String	✓	Viewer user ID

11. Category Router

All endpoints prefixed with `category.*` via tRPC.

11.1 Get All Categories

Field	Value
Endpoint	<code>/api/trpc/category.getAll</code>
Method	<code>query</code> (GET)
Auth	Bearer Token (Clerk Session)
Description	Returns all categories sorted by project count descending.

Request Body — None

Success Response — `200 OK`

```
{
  "success": true,
  "message": "Successfully get 5 categories",
  "data": [
    { "id": "...", "slug": "web-development", "title": "Web Development", "count_projects": 42 }
  ]
}
```

11.2 Get One Category

Field	Value
Endpoint	<code>/api/trpc/category.getOne</code>
Method	<code>query</code> (GET)
Auth	Bearer Token (Clerk Session)
Description	Returns a single category by ID or slug.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id</code>	String	✗	Category ID
<code>slug</code>	String	✗	Category slug

12. Pin Router

All endpoints prefixed with `pin.*` via tRPC.

12.1 Pin Project

Field	Value
Endpoint	<code>/api/trpc/pin.pin</code>
Method	<code>mutation</code> (POST)
Auth	Bearer Token (Clerk Session)
Permission	Project owner or accepted collaborator
Description	Pins a project to the user's profile.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id_project</code>	String	✓	Project ID or slug
<code>id_user</code>	String	✓	User ID

Errors

Code	Error	Description
<code>404 NOT_FOUND</code>	—	Project not found or no permission
<code>409 CONFLICT</code>	—	Already pinned

12.2 Unpin Project

Field	Value
Endpoint	<code>/api/trpc/pin.unpin</code>
Method	<code>mutation</code> (POST)
Auth	Bearer Token (Clerk Session)
Permission	Project owner or accepted collaborator
Description	Removes a pinned project from the user's profile.

Request Body (JSON)

Parameter	Type	Required	Description
<code>id_project</code>	String	✓	Project ID or slug
<code>id_user</code>	String	✓	User ID

Errors

Code	Error	Description
404 NOT_FOUND	—	Project not found or pin not found

13. REST API Endpoints

These endpoints use standard HTTP REST conventions via Next.js App Router.

13.1 Upload Profile Image

Field	Value
URL	POST /api/profile/editProfile
Auth	Bearer Token (Clerk Session)
Content-Type	multipart/form-data
Description	Uploads a new profile image. Deletes the old image if provided. Stores files under {userId}/{folder}/.

Request Body (Form-Data)

Parameter	Type	Required	Description
file	File	✓	Image file to upload
folder	String	✓	Storage folder name
oldImagePath	String	✗	Path to old image for deletion

Request Example

```
curl -X POST https://your-app.com/api/profile/editProfile \
  -H "Authorization: Bearer YOUR_CLERK_TOKEN" \
  -F "file=@photo.jpg" \
  -F "folder=profile" \
  -F "oldImagePath=user_123/profile/old.jpg"
```

Success Response — 200 OK

```
{ "newImagePath": "user_123/profile/1714816800000-photo.webp" }
```

Errors

Status	Error	Description
400	"File and folder are required."	Missing required fields
401	"Unauthorized"	No valid session
500	"Failed to edit profile image."	Upload failure

13.2 Create Project (REST)

Field	Value
URL	POST /api/project/create
Auth	Bearer Token (Clerk Session)
Content-Type	multipart/form-data
Description	Creates a project with image uploads. Uploads images to Supabase Storage, then calls <code>trpc.project.create</code> internally.

Request Body (Form-Data)

Parameter	Type	Required	Description
title	String	✓	Project title
id_category	String	✓	Category ID
content	String	✓	HTML content body
image1	File	✓	Primary image (required)
image2 – image5	File	✗	Additional images
link_figma	String	✗	Figma link
link_github	String	✗	GitHub repo link
collaborators	JSON String	✗	Array of <code>{id, name, username, photo_profile?}</code>

Errors

Status	Error	Description
400	"Name, id_category and content are required"	Missing required fields
500	"Failed to upload image"	Image upload failure
500	"Failed to create project"	Database creation failure

13.3 Edit Project (REST)

Field	Value
URL	<code>POST /api/project/edit</code>
Auth	Bearer Token (Clerk Session)
Content-Type	<code>multipart/form-data</code>
Description	Edits project metadata (title, content, category, links). Calls <code>trpc.project.edit</code> internally.

Request Body (Form-Data)

Parameter	Type	Required	Description
<code>id</code>	String	✓	Project ID
<code>id_category</code>	String	✗	New category ID
<code>title</code>	String	✗	New title
<code>content</code>	String	✗	New content
<code>link_figma</code>	String	✗	Figma link
<code>link_github</code>	String	✗	GitHub link

Errors

Status	Error	Description
<code>400</code>	<code>"Project ID and user ID are required"</code>	Missing IDs
<code>404</code>	<code>"Project not found or access denied"</code>	Not found or not owner
<code>500</code>	<code>"Failed to edit project"</code>	Database error

13.4 Upload Project Images

Field	Value
URL	<code>POST /api/project/uploadImage</code>
Auth	Bearer Token (Clerk Session)
Content-Type	<code>multipart/form-data</code>
Description	Uploads one or more images for a project. Files stored under <code>{userId}/{folder}/</code> .

Request Body (Form-Data)

Parameter	Type	Required	Description
<code>folder</code>	String	✓	Storage folder (e.g. <code>"project"</code>)
<code>files[0]</code> , <code>files[1]</code> , ...	File	✓	Image files (at least one required)

Success Response — 200 OK

```
{ "filePaths": ["user_123/project/1714816800000-img1.webp", "user_123/project/1714816800001-img2.webp"] }
```

Errors

Status	Error	Description
400	"Folder is required."	Missing folder
400	"No files provided."	No files attached
401	"Unauthorized"	No valid session
500	"Failed to upload images."	Upload failure

Global Error Reference

All tRPC endpoints may return the following common errors:

HTTP Status	tRPC Code	Description
401	UNAUTHORIZED	Authentication token is missing or invalid
403	FORBIDDEN	Insufficient permissions for the action
404	NOT_FOUND	Requested resource does not exist
409	CONFLICT	Duplicate resource (already exists)
400	BAD_REQUEST	Invalid or missing input parameters
500	INTERNAL_SERVER_ERROR	Unexpected server error

Enum Reference

`genderType`

MALE | FEMALE

`notifType`

FOLLOW | LIKE_PROJECT | LIKE_COMMENT | COMMENT_PROJECT | COMMENT | REPLY_COMMENT |
COLLABORATION

ownershipType

OWNER | COLLABORATOR

collabStatusType

PENDING | ACCEPTED | REJECTED

End of documentation.